

Crowd Flow Optimiser.

Simulate a venue, predict where the crowd will jam, and route around it — before the queue becomes a crush.

5

runtimes in one stack

87%

of bottlenecks caught
before they form

51 → 0

zone-ticks above critical

175

automated tests

Contents

WHAT IS IN THIS DOCUMENT

1	The problem	7	Proving it works
2	What the system does	8	Floor plan to walkable graph
3	Architecture	9	Accounts, access and privacy
4	What is an algorithm, what is AI	10	Testing and delivery
5	The crowd simulation	11	Running it
6	The AI layer	12	Known limits and what is next

How to read this

The system is four cooperating services and a mobile client. Sections 3 to 8 explain how a crowd gets from a floor plan to a routed evacuation; sections 9 to 12 cover how it is secured, tested, shipped and where it stops. Every number quoted is produced by code in the repository and is reproducible with the commands in section 11.

THE ONE CLAIM WORTH HOLDING ONTO

Most crowd tools report what *is* happening. This one runs a second, hidden simulation of the same crowd with rerouting switched off, on the same random seed, and compares them. The before/after figure is therefore a measurement rather than an estimate — that pairing is the heart of the project.

1 • The problem

WHY CROWD CRUSHES ARE A ROUTING FAILURE

Crowds bunch at gates, food counters and exits without warning. The dangerous moment is not when a venue is full. It is when one zone fills while the zones beside it stay empty, because everybody was following the same shortest path to the same door. By the time density is visible on a CCTV wall, the people who could have been diverted are already inside the jam.

The information needed to prevent it usually exists and is simply never joined up. The venue knows its own floor plan. Ticketing knows roughly how many people are arriving and when. Stewards know which corridors are moving. What nobody has is a way to put those together and answer the only question that matters a few minutes ahead of time: *where will the pile-up be, and where should these people go instead?*

What this project builds

A venue is uploaded as a floor plan and becomes a graph of zones and walkable connections. A crowd is simulated moving through it, person by person. Density is measured per zone every tick, a model predicts which zones are about to become dangerous, and the system diverts people around them, then proves the diversion helped by running the same crowd again without it.

Who uses it

ROLE	WHAT THEY GET
Attendee	Types the venue code from the entrance signage and gets the map with a route out that is coloured by live congestion, and planned <i>around</i> the crowd rather than through it.
Organiser	Traces a floor plan into a graph, runs the session, watches occupancy fill zone by zone, and reads a ranked crowd-safety panel saying which zone is becoming dangerous and what to do about it.
Operations	Every session on the platform on one board, with aggregate figures and the alert feed.

2 · What the system does

THE LOOP, END TO END

1. **A venue becomes a graph.** An organiser uploads a 2D floor plan. Computer vision traces the walkable space into zones (GATE, WALKWAY, CONCESSION, SEATING, EXIT) joined by edges carrying a length and a width. The operator corrects it on screen; the corrected graph is validated again before anything runs on it.
2. **A crowd is simulated on it.** Agents arrive at gates on a schedule and move toward their destinations under a social force model, ten times a second.
3. **Density is measured.** Every zone reports occupancy against capacity, tagged rising, flat or falling over a rolling window.
4. **Risk is predicted.** A graph neural network takes the current densities and the venue structure and returns, per zone, the risk of crossing the critical line within the next few ticks.
5. **People are rerouted.** Newly alerting zones trigger a Dijkstra search for an alternative, and a small language model turns the numbers into an instruction a steward can act on.
6. **The result is proved.** A hidden twin of the same run, same seed, rerouting off, ticks in lockstep. The summary compares them.

What an attendee sees

A route drawn segment by segment in the colour of the zone it enters (blue clear, amber moderate, orange heavy, red severe), with a banner that says when the route has been diverted and what the diversion cost in metres. The route is recomputed in the browser from the live density arriving on the session socket five times a second, so it responds to the crowd without a request per attendee per tick.

A DELIBERATE DESIGN DECISION

A crowded zone is expensive to walk into, not forbidden. The cost of entering a zone rises with the fourth power of its density, so a short hop through a busy area still beats a long detour — which is what a navigation app does and what an evacuating crowd should do. Only genuinely crushed zones, above 94% of safe capacity, are treated as walls.

3 • Architecture

FOUR SERVICES, AND A FIFTH CLIENT



Each tier degrades rather than failing. The frontend renders signed-out, the backend keeps ticking and broadcasting measured density with the AI service down, and the AI service answers from its offline scorer with no model downloaded and no token set.

Why the AI layer is a separate service

The models are Python and the simulation is Java, and forcing either into the other's runtime would have cost more than the HTTP hop. Splitting them also produces the property the demo depends on: the AI call is always made *off* the tick thread, so a slow or dead /analyze costs a stale risk number rather than a frozen simulation. A session that loses its AI layer keeps running and keeps broadcasting real measured density.

State, and what survives a restart

DATA	WHERE	LIFETIME
Accounts	H2 file locally, Postgres in cloud	Permanent, Flyway-migrated
Venues	One JSON file per venue code	Permanent; a printed code must still work tomorrow
Sessions	In memory	Die with the process, by design
Attendee positions	Zone id + expiry, in the session	30 seconds without a fresh report

4 • What is an algorithm, what is AI

A DELIBERATE SPLIT, NOT A DEFAULT

Forcing machine learning into the deterministic parts of this problem would have cost time and bought nothing. Each piece uses the cheapest technique that actually answers its question.

PIECE	APPROACH	WHY
Crowd movement per tick	Social force model over a capacitated graph	Deterministic, fast, debuggable. A crowd that moves wrongly is a bug you can step through.
Reroute suggestion	Dijkstra	Provably shortest. There is nothing to train and nothing that could be more correct.
Where congestion spreads next	Graph neural network	Needs the graph structure. A per-zone threshold cannot see a neighbour about to push its crowd into you.
Turning numbers into an instruction	Small instruct model	Operators read sentences, not density vectors.
Floor plan → walkable graph	OpenCV skeletonisation, optional VLM pass	Geometry, not perception. The vision model only labels what the CV pass already found.

WHY A GNN AND NOT A THRESHOLD

A threshold answers "is this zone busy". The question that prevents a crush is "is this zone *about to become* busy", and the answer usually lives in its neighbours: a concourse at 60% with two full stands feeding into it is in far more trouble than a concourse at 75% with empty ones. That relationship is the graph's structure, which is exactly what a message-passing network reads and a per-zone rule cannot.

5 • The crowd simulation

HELBING–MOLNÁR OVER A CAPACITATED GRAPH

Agent movement

Each agent is pulled toward its next waypoint by a driving term, pushed away from other agents by an exponential repulsion, and pushed off walls by the same exponential. Agents slow and swerve rather than overlap, which is what collision handling means in a crowd model. People do not bounce, they hesitate and go around.

Everything works in **venue layout units, one integration step per tick**, not metres and seconds. The published Helbing constants assume SI units, so the constants here are calibration knobs rather than physical ones, and they bind from configuration so a crowd can be retuned against a new layout without a rebuild. Being explicit about this matters: quoting SI constants for a model that does not run in SI would be a claim the code does not support.

One tick, in order

1. The scheduler fires — every 100 ms in agent mode.
2. Exits drain. Everyone else advances toward their destination as far as downstream capacity and edge throughput allow. **People who cannot move stay put**, which is what makes density climb rather than being imposed.
3. Arrivals enter at the gates on the schedule's curve.
4. Risk is predicted from the current densities and the graph.
5. Zones over threshold are tagged rising, flat or falling from the last five ticks.
6. For each *newly* alerting zone: find a reroute, then generate an advisory. Alerts fire on severity change only, or the feed floods and becomes unreadable.
7. The new state is pushed to every WebSocket watching the run.

Why congestion limits flow at the destination

Congestion slows people *entering* a crowded space; it does not paralyse the space they are leaving. A packed gate still empties at the corridor's rate. Modelling it the other way round, scaling how fast people leave by their own zone's density, makes jams self-locking: they never clear, no routing change can help, and the model quietly proves that nothing works. That dead end is avoided on purpose.

Broadcast economics

The clock runs at 10 Hz and frames are pushed at 5 Hz, which halves socket traffic without the map looking choppy. Agent positions per frame are capped and sampled, with the true crowd size still reported alongside, so a 5,000-person venue does not put 5,000 coordinates on the wire twice a second.

6 • The AI layer

TWO MODELS, AND A FALLBACK THAT ALWAYS ANSWERS

Congestion GNN

A graph neural network trained on synthetic runs of the simulator, published to the Hugging Face Hub as `abhi1005/congestion-gnn`. It takes the venue graph and the current per-zone densities and returns the risk that each zone crosses the critical line within the prediction horizon.

The honest way to score it is the part worth reading carefully. Measured across *all* zone-ticks above the line, a persistence baseline ("assume every zone stays as it is") already scores about 92% recall, because most zones that are critical in five ticks are already critical now. That number measures reporting, not prediction, and quoting it would be misleading.

The real claim is **onset**: zones below the line now that cross it within the horizon. Persistence scores exactly zero on that by construction. On held-out runs:

	CAUGHT	RECALL	PRECISION
Congestion GNN	1,421 of 1,630	87%	95%
Persistence baseline	0 of 1,630	0%	—

THE NUMBER TO QUOTE

The model catches 87% of bottlenecks before they form, at 95% precision. Not the 99% that the easier framing would have produced.

Operator advisory

A small instruct model turns a density vector into a sentence a marshal can act on. It is used as-is rather than fine-tuned. Converting numbers to fluent instructions is precisely what small instruct models already do well.

It is also the component most likely to lie, so it is constrained rather than trusted: an advisory naming a zone that was not in the prompt is **rejected outright** and the deterministic template is used instead. Fluent and wrong is worse than plain and right when somebody is about to move a crowd on the strength of it.

The fallback ladder

Every inference step chooses its path independently, so a working GNN with a missing advisory model still gives real risk scores with templated prose.

PATH	USED WHEN
Hosted inference	A token and endpoint URL are configured
In-process models	The model is present and its library installed
Offline scorer	Always available — no download, no network, no token

The health endpoint names whichever path actually answered, so "why is the advisory generic" takes one request to diagnose rather than an hour mid-demo.

7 • Proving it works

THE HIDDEN TWIN

Claiming that rerouting helps is easy. Measuring it requires the counterfactual, so starting a run with rerouting enabled also starts a hidden twin: the same venue, the same crowd, the same random seed, rerouting off, ticked in lockstep. The summary compares the two. The difference is real simulation output rather than an estimate.

The two levers

Both match what the advisories actually tell staff to do:

1. **Spill sideways.** People may move *laterally* (to a zone the same hop-distance from the exit) when the direct route is full, instead of queueing. Never backwards: sending people back toward the gate is not a reroute, it is a crush.
2. **Hold intake.** A gate stops accepting arrivals just below the critical threshold, so people wait outside rather than pack into the gate.

Result on the sample venue

51 → 0

Zone-ticks above critical density. The headline number.

85%

Peak density, against 100% untreated.

same seed

Identical crowd and arrival curve on both sides.

Reading the numbers honestly

Time spent above critical is the headline, and the other candidates mislead on their own. Peak density gets pinned at 100% by any single undersized kiosk regardless of routing. Bottleneck *count* actually *rises* when a crowd is successfully spread out, because more zones briefly touch the threshold. Time in the red is what routing genuinely moves.

The summary also declines to compare throughput. The untreated run usually gets more people through, by packing gates several times over capacity, which is precisely the thing being prevented. Reporting it as a win for the untreated side would be true arithmetic and a false claim.

REAL ATTENDEES ARE EXCLUDED, DELIBERATELY

Attendees using the mobile app raise the density an operator sees, but they are kept out of the two compared numbers. The twin has no real attendees and cannot have any — it is a second simulation of the same seed, not a second venue people can walk into. Counting a phone standing in a gate would add it to the optimised side and nowhere else, and the summary would report that rerouting had made the venue *worse*. One test, `realAttendeesRaiseLiveDensityButNeverTheBaselineNumbers`, is the executable form of that sentence.

8 · Floor plan to walkable graph

WHERE A VENUE COMES FROM

An organiser should not have to hand-author a graph. They upload a 2D plan, and the pipeline turns it into one. The graph, never the image, is the source of truth. No PNG, no rendered map and no model output is canonical; everything downstream reads the one graph object.

The stages

1. **Normalise and mask.** The plan is scaled to a working canvas and reduced to a walkable/not-walkable mask.
2. **Rooms, optionally.** Enclosed rooms are separated from circulation space, or corridors get traced straight through them. Off for arenas, on for anything with rooms.
3. **Skeletonise.** The walkable area is thinned to a centre-line network — the expensive stage, and the reason the tuning preview stops just before it.
4. **Build the graph.** Junctions become nodes, the runs between them become edges carrying length and width.
5. **Label.** A vision-language model optionally names zones by reading the plan's own text. It only labels what the geometry already found; it never invents a zone.
6. **Validate and repair.** Deterministic checks with automatic repair where the fix is unambiguous.

The human is in the loop, and the loop is enforced

The operator edits the traced graph on screen, and the confirm step **re-runs validation on the edited graph** rather than trusting the client. An operator dragging nodes around can disconnect a gate from every exit just as easily as the tracer can, and a graph where somebody has no route out is the one failure that must never reach a simulation. The start button stays disabled while errors remain.

GRACEFUL ABSENCE

The vision stack is a separate dependency set. If it is not installed the service still starts and `/analyze` is unaffected; the health endpoint simply reports layout tracing as unavailable, so the UI can say so instead of letting an operator upload a plan and receive a 404.

9 · Accounts, access and privacy

WHO MAY CALL WHAT, AND WHAT IS NEVER COLLECTED

Three providers, one code path

Every provider reduces to the same question — is this token validly signed by someone I trust, and who does it say the caller is? — so they differ only in the key used to check the signature. Each incoming token is offered to every enabled verifier in turn and the first that recognises it wins. All three can be active simultaneously, and a provider with no configuration reports itself disabled and is skipped.

PROVIDER	SIGNING	SIGN-UP HANDLED BY
Local	HS256, our secret	This backend
Supabase	HS256, project JWT secret	Supabase
Firebase	RS256, Google rotating keys	Firebase

Admin is not self-service

Reads are public so the marketing pages and the live map work signed-out; writes need a role. Administrator cannot be requested at registration and cannot be reached by signing in at the admin door; both answer 403. The grant comes from a configured allowlist applied at every registration and login, which makes it reversible from outside the database in both directions. Critically, the same allowlist now governs federated sign-ins: a token from an external provider claiming an administrator role is capped at organiser unless its address is on the list.

Defence in depth

- **Uniform answers.** Password reset returns the same response for a registered address, an unknown one and a disabled one — otherwise the endpoint becomes a free tool for discovering which emails hold accounts. Login likewise cannot distinguish "no such user" from "wrong password".
- **Rate limiting ahead of authentication.** Token buckets refuse a login attempt before it costs the server the password hash it cannot decline to compute, which is what made an unlimited guess rate and a denial of service the same request.
- **The deployment cannot boot insecure.** The cloud profile refuses to start on the development JWT secret or on one too short to be one.
- **Service-to-service authentication.** The AI layer requires a shared secret, compared in constant time, so an exposed port is not free access to endpoints that run computer vision on whatever they are handed.
- **Browser hardening.** A content security policy, framing and sniffing protections, and inline-only avatars, since a remote avatar URL would let a profile decide which third-party host every viewer's browser calls.

What is deliberately not collected

The attendee experience is built to need as little as possible, and the limits are architectural rather than promised:

- **A zone, not a position.** A GPS fix is resolved to a zone at the ingest boundary and the coordinates are discarded. The session object *has no field* to put them in. The fix is echoed back to the phone that sent it, so it can draw its own dot, and to nobody else.
- **A fix that cannot support its claim is thrown away.** If the reported accuracy circle is larger than the zone it lands in, nobody is placed. The app never draws a position it does not have.
- **Foreground only.** No background location permission is requested, and an attendee who closes the app ages out of the venue within 30 seconds.
- **Anonymous and session-scoped.** The identifier is a UUID the app generates and keeps to itself. There is no account and nothing to join it to.

10 • Testing and delivery

WHAT HAS TO PASS BEFORE ANYTHING MERGES

102

Backend tests, through the real security filter chain

49

AI service tests, including the model contract

24

Frontend unit tests, plus render and portal smokes

The layers, and what each is for

LAYER	WHAT IT CATCHES
Backend unit & API	Access rules driven through the real filter chain, simulation mechanics, walker ingest and expiry, the before/after invariant.
AI service	Feature-column contract between training and serving, degradation when a model endpoint is dead, the layout pipeline, the service token gate.
Frontend unit	Routing and the congestion cost model, credential rules, venue codes.
Render smoke	Every route mounts. A production build only proves the module graph resolves — it cannot catch a component that is undefined at the call site or a crash inside render, which are blank-screen bugs in the browser.
Stack smoke	One attendee's journey across all three services: register, upload a venue, run a session, report a position, read the frame.

WHY THE STACK TEST EXISTS

Every other suite runs one service with its neighbours stubbed. That is correct for a unit test, and it is exactly why nothing caught a mismatch *between* services. A renamed field, a security rule that refuses a call the frontend has always made, a migration that did not run: each passes every unit test and breaks the product. The stack test also asserts the negatives, that an anonymous venue upload is refused and that the attendee path still works *without* a token. It found three field mismatches while it was being written.

Pipelines

Two, running the same commands: GitHub Actions gates every pull request, and a containerised Jenkins pipeline mirrors it. Both run cheapest-signal-first — four test jobs in parallel that fail in seconds, with the full container stack build waiting behind them rather than spending fifteen minutes on a commit a unit test already rejected. The stack job brings the compose environment up, waits for every container to report healthy, and then runs the end-to-end pass against it.

11 • Running it

FROM A CLEAN CHECKOUT

Everything, in one command

```
docker compose up --build
```

Brings up Postgres, the AI service, the backend and the built frontend together at `localhost:5173`. This runs the backend under its cloud profile against Postgres, which is the shape a real deployment has rather than the shape a laptop has. The same Flyway migrations run either way, which is the point.

Or as three processes, which is the better development loop

Start them in any order; each degrades gracefully while the others are down. The AI service needs no token and no model — it answers from its offline scorer on a clean checkout, which is the property that makes the demo survive bad conference wifi. The backend writes to a local H2 file, so no database install is required and accounts survive a restart.

Reproducing the numbers in this document

```
backend    ./mvnw test                # 102 tests
ai-service python -m pytest tests -q    # 49 tests
frontend   npm test && npm run test:render
stack      docker compose up -d --wait && ci/smoke-stack.sh
model      python gnn/evaluate.py --checkpoint out/congestion_gnn.pt
```

Configuration philosophy

Nothing required has a default that needs a network or a token. Every secret is absent from the repository and supplied by environment variable or an ignored local file, and every one of them has a committed example showing its shape. The admin allowlist ships *empty* on purpose. This is a public repository, and a real address in it would publish both who holds administrator access and their inbox. Empty means nobody has it, which is the safe way to be wrong.

12 · Known limits and what is next

STATED RATHER THAN DISCOVERED

Limits

LIMIT	CONSEQUENCE, AND THE FIX
Sessions live in memory	A restart loses running simulations; accounts and venues survive. Durable runs are a change to the session manager, not a config switch.
Rate limiting is per-process	Two instances behind a load balancer allow twice the configured rate. A shared store, or a limit at the proxy, makes it exact.
Tokens do not refresh	Twelve hours, then a fresh login. Refresh tokens mean a second table and a rotation endpoint.
Attendee position is zone-level and self-declared	Deliberate. The system simulates a crowd; it does not track anyone's phone, and the interface says so rather than drawing a false accuracy circle.
Model trained on synthetic runs	The simulator generates its training data, so the GNN learns this model's dynamics. Real venue telemetry is the honest next input.

Next

- **Real telemetry.** Turnstile counts and anonymised WiFi association counts are already zone-level, which is exactly the shape the ingest boundary accepts. Nothing in the pipeline needs to change to take them.
- **Scale the fan-out, not the API.** The WebSocket broadcast, rather than the HTTP surface, is what stands between this and tens of thousands of concurrent viewers.
- **Schedule awareness.** Arrival curves are configured per run; the event schedule format already exists and could drive them directly, so a set change or a full-time whistle is anticipated rather than observed.
- **Browser-level tests.** Rendering is smoke-tested server-side; a real driver over a running stack would close the last gap.

THE IDEA IN ONE PARAGRAPH

A venue is a graph, a crowd is a simulation over it, and a crush is a routing failure that can be seen coming. Classical algorithms handle the parts that are deterministic (movement and shortest paths) and machine learning handles the two that are genuinely not: where congestion will spread through the structure, and how to say that in a sentence somebody can act on. The hidden twin is what turns the whole thing from a claim into a measurement.